

Zadanie 1: Wprowadzenie do programowania w C#

Tematy: Kompilator, interpreter, debugger; Biblioteki, kompilacja i uruchomienie programów; Instalacja środowiska Visual Studio, charakterystyka języka C#; Struktura projektu i programu; Proste i złożone typy danych; Zmienne; Operacje na zmiennych: wejścia, wyjścia, arytmetyczne, logiczne.

Zacznijmy naszą przygodę z programowaniem w C#. To zadanie będzie Twoim pierwszym krokiem w tworzeniu interaktywnych programów. Wyobraź sobie, że komputer to potężny kalkulator, który potrafi wykonywać skomplikowane obliczenia na Twoje komendy. Dzięki językowi C# będziesz mógł przekazywać komputerowi precyzyjne instrukcje, co pozwoli Ci rozwiązywać różnorodne problemy.

C# – wszechstronny język dla nowoczesnych programistów

C# to niezwykle popularny, obiektowy język programowania stworzony przez Microsoft, który zdobył uznanie programistów na całym świecie. Jego czytelna składnia, oparta na najlepszych praktykach z innych języków, takich jak C++ i Java, sprawia, że jest łatwy w nauce i użyciu. C# jest ściśle związany z platformą .NET, która zapewnia bogaty zestaw narzędzi i bibliotek, umożliwiających tworzenie różnorodnych aplikacji – od prostych programów konsolowych po zaawansowane systemy biznesowe. Dzięki swojej wydajności, bezpieczeństwu i możliwości tworzenia wieloplatformowych aplikacji, C# jest jednym z najczęściej wybieranych języków do realizacji projektów zarówno na dużą, jak i małą skalę.

W tym zadaniu skupimy się na podstawowych pojęciach:

Kompilator: Program, który przekształca kod źródłowy (np. napisany w C#) na kod maszynowy, czyli zrozumiały bezpośrednio dla procesora. W Visual Studio Kompilator C# jest zintegrowany ze środowiskiem i automatycznie uruchamiany podczas procesu budowania projektu.

Proces kompilacji: Sprawdza poprawność składniową kodu, konwertuje go na kod pośredni (IL), a następnie na kod maszynowy. Proces kompilacji składa się z następujących kroków:

- **Analiza składniowa:** Sprawdzenie, czy kod jest poprawny gramatycznie.
- **Generowanie kodu pośredniego (IL):** Przekształcenie kodu źródłowego na formę zrozumiałą dla maszyny wirtualnej (Common Language Runtime).
- **Kompilacja JIT (Just-In-Time):** Tłumaczenie kodu pośredniego na kod maszynowy tuż przed wykonaniem.
- **Optymalizacja:** Kompilatory C# stosują różne techniki optymalizacji kodu, aby zwiększyć jego wydajność.

Interpreter: Program, który wykonuje instrukcje programu bezpośrednio, bez uprzedniej kompilacji do kodu maszynowego. Choć C# jest językiem kompilowanym, niektóre środowiska (np. interaktywne konsole) umożliwiają interpretację krótkich fragmentów kodu.

Debugger: Narzędzie służące do śledzenia wykonania programu krok po kroku, ustawiania punktów przerwania, inspekcji wartości zmiennych itp. W Visual Studio zaawansowany debugger pozwala na szczegółową analizę kodu podczas debugowania, w tym wizualizację zmiennych, wątków i innych elementów aplikacji. Funkcjonalności debuggера to ustawianie punktów przerwania, krokowanie, inspekcja zmiennych, wywoływanie funkcji w czasie debugowania.

Biblioteki: Zbiory gotowych funkcji i klas, które można wykorzystać w swoich programach, oszczędzając czas i wysiłek. W C# bogata biblioteka klas .NET Framework dostarcza szeroki zakres funkcjonalności, od obsługi plików po tworzenie interfejsów użytkownika. Podstawowe rodzaje bibliotek to: Standardowe biblioteki .NET, biblioteki zewnętrzne (np. NuGet), biblioteki własne.

Zmienne: To są jak pojemniki, w których przechowujesz dane, takie jak liczby, słowa czy bardziej złożone informacje. Wyobraź sobie, że zmienna to szuflada, do której wkładasz różne przedmioty.

Typy danych: Każda zmienna ma określony typ, który określa, jakie wartości może przechowywać. Na przykład, zmienna typu `int` może przechowywać liczby całkowite, a zmienna typu `string` może przechowywać tekst.

Operatory: To są symbole, które służą do wykonywania różnych operacji na danych, takich jak dodawanie, odejmowanie, mnożenie, dzielenie czy porównywanie.

Instrukcje: To są polecenia, które mówią komputerowi, co ma zrobić. Na przykład, instrukcja `Console.WriteLine()` służy do wyświetlenia tekstu na ekranie.

Dlaczego warto zacząć od zadań konsolowych? Aplikacje konsolowe są świetnym punktem wyjścia dla początkujących, ponieważ pozwalają skupić się na logice programowania bez konieczności zajmowania się tworzeniem graficznego interfejsu użytkownika. Dzięki temu szybciej zrozumiesz podstawowe zasady programowania w C#.

Jesteś gotowy, aby zacząć tworzyć swoje pierwsze programy? Pamiętaj, że programowanie to jak nauka nowego języka. Na początku może wydawać się skomplikowane, ale z każdym kolejnym zadaniem będziesz coraz bardziej wprawny.

Opis instalacji Visual Studio do programowania w C#

Visual Studio to potężne środowisko programistyczne (IDE), które umożliwia tworzenie różnorodnych aplikacji, od prostych programów konsolowych po złożone aplikacje desktopowe. W tym samouczku przeprowadzimy Cię przez proces instalacji Visual Studio, skonfigurowanego do pracy z językiem C#.

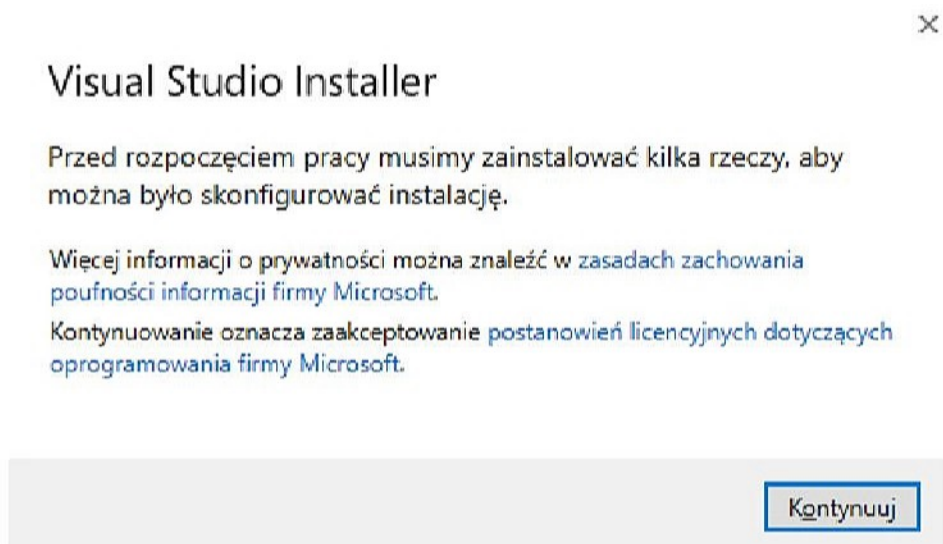
Krok 1: Pobranie instalatora

Odwiedź oficjalną stronę Visual Studio:

<https://visualstudio.microsoft.com/pl/downloads/>

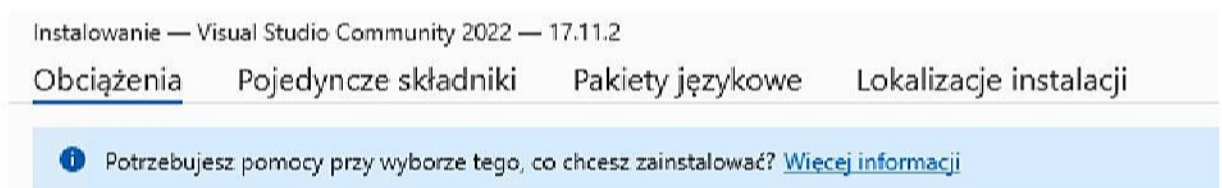
Wybierz odpowiednią wersję: Wybierz wersję Visual Studio, która najlepiej odpowiada Twoim potrzebom. Dla większości użytkowników odpowiednia będzie darmowa wersja Community.

Uruchom instalator: Po zakończeniu pobierania uruchom plik instalacyjny.



Krok 2: Wybór obciążeń

Ekran wyboru obciążeń: W instalatorze pojawi się okno, w którym możesz wybrać obciążenia (ang. workloads) – zestawy narzędzi i funkcji, które chcesz zainstalować.



Wybór dla aplikacji konsolowych:

.NET desktop development: Wybierz to obciążenie, aby uzyskać wszystkie niezbędne narzędzia do tworzenia aplikacji konsolowych w C#.



Programowanie aplikacji klasycznych dla platformy .NET ☒
Twórz aplikacje WPF i Windows Forms oraz aplikacje konsolowe przy użyciu języków C#, Visual Basic i F# za p...

Wybór dla aplikacji desktopowych:

.NET desktop development: To obciążenie również zawiera narzędzia do tworzenia aplikacji desktopowych.



Opracowywanie aplikacji dla systemu Windows ☒
Twórz aplikacje dla platformy systemu Windows przy użyciu biblioteki WinUI z językiem C# lub opcjonalnie C+...

ASP.NET and web development: Jeśli planujesz tworzyć aplikacje internetowe, możesz również dodać to obciążenie.



Opracowywanie zawartości dla platformy ASP.NET i sie... ☒
Twórz aplikacje internetowe dla wielu platform przy użyciu technologii ASP.NET Core, ASP.NET, HTML/JavaScript i k...

Krok 3: Dodatkowe składniki (opcjonalnie)

Indywidualne składniki: Możesz dodatkowo wybrać poszczególne składniki, takie jak narzędzia do debugowania, analizy kodu czy integracji z systemami kontroli wersji.

Krok 4: Instalacja

Zainstaluj: Kliknij przycisk "Zainstaluj", aby rozpocząć proces instalacji. Czas instalacji może zależeć od wybranego obciążenia i specyfikacji Twojego komputera.



Zaloguj się

Tworzenie konta

[Na razie pomiń to.](#)

Krok 5: Uruchomienie Visual Studio

Po zakończeniu instalacji uruchom Visual Studio.



Ustawienia projektowania

Ogólne

Wybierz motyw koloru

☐ Ciemny

☒ Jasny

☐ Niebieski

☐ Niebieski (dodatkowy k...)

Uruchom program Visual Studio

Tworzenie pierwszego projektu

Aplikacja konsolowa to najprostszy rodzaj programu, który komunikuje się z użytkownikiem za pomocą wiersza poleceń. Nie posiada graficznego interfejsu użytkownika (GUI), a dane wejściowe i wyjściowe są przekazywane poprzez standardowe strumienie wejścia/wyjścia (np. klawiaturę i ekran).

Różnice między .NET Framework a .NET Core:

Cecha	.NET Framework	.NET Core
Cel	Tradycyjne aplikacje Windows, serwery, usługi webowe	Wieloplatformowe aplikacje, mikroserwisy, aplikacje konsolowe, webowe, mobilne
Model wdrożenia	Side-by-side, wymaga instalacji .NET Framework	Self-contained lub zależny od frameworka, wdrożenie jako pojedynczy plik lub folder
Wydajność	Zoptymalizowany pod kątem dużych, złożonych aplikacji	Lekki, szybki start, wysoka wydajność
Wsparcie dla platform	Windows	Windows, macOS, Linux, Docker
Otwarte źródło	Nie	Tak
Nowoczesne technologie	Mniejsze wsparcie dla nowych technologii	W pełni wspiera najnowsze technologie .NET

Kiedy wybrać który szablon?

.NET Framework:

Zalety: Dojrzała platforma, duża baza bibliotek, idealna dla istniejących aplikacji Windows.

Kiedy wybrać:

- Jeśli masz już istniejący kod w .NET Framework i chcesz go rozwijać.
- Jeśli potrzebujesz maksymalnej kompatybilności z innymi aplikacjami Windows.
- Jeśli tworzysz duże, złożone aplikacje desktopowe.

.NET Core:

Zalety: Wysoka wydajność, wieloplatformowość, łatwe wdrożenie, otwartość, ciągły rozwój.

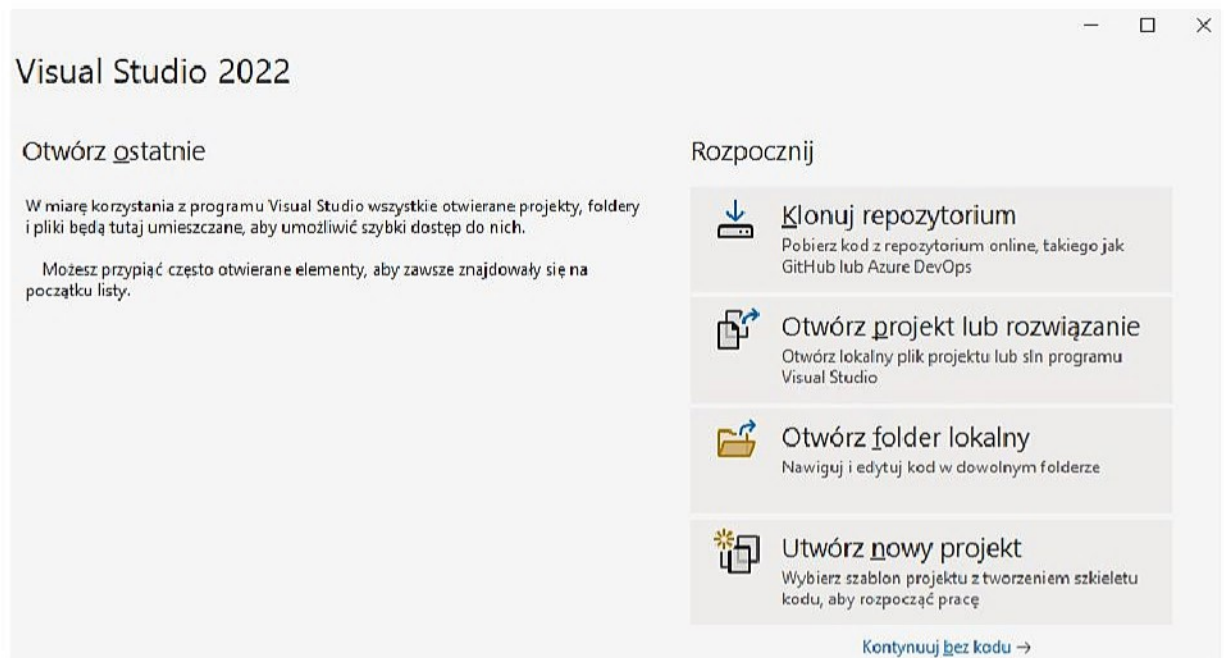
Kiedy wybrać:

- Jeśli tworzysz nowe aplikacje od podstaw.
- Jeśli potrzebujesz wysokiej wydajności i skalowalności.
- Jeśli chcesz tworzyć aplikacje działające na różnych platformach.

- Jeśli chcesz korzystać z najnowszych technologii .NET.

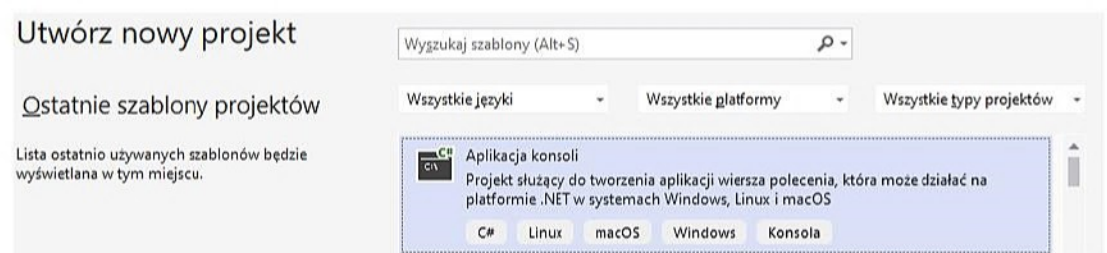
Oba szablony pozwalają na tworzenie aplikacji konsolowych w C#, ale różnią się pod względem docelowej platformy, wydajności i wsparcia. Wybór odpowiedniego szablonu zależy od konkretnych wymagań projektu. Jeśli zaczynasz przygodę z .NET, **.NET Core** jest zazwyczaj lepszym wyborem ze względu na swoją nowoczesność, wydajność i elastyczność.

1. **Nowy projekt:** Po uruchomieniu Visual Studio wybierz "Utwórz nowy projekt".



2. **Wybierz szablon:**

Aplikacja konsolowa: Wybierz szablon "Aplikacja konsolowa (.NET Framework)" lub "Aplikacja konsolowa (.NET Core)", w zależności od preferencji.



Aplikacja Windows Forms: Wybierz szablon "Aplikacja systemu Windows Forms (.NET Framework)" lub ".NET Framework" dla aplikacji desktopowych z tradycyjnym interfejsem.

Aplikacja WPF: Wybierz szablon "Aplikacja WPF (.NET Framework)" lub ".NET" dla aplikacji desktopowych z nowoczesnym interfejsem.

3. **Nadaj nazwę projektowi:** Podaj nazwę dla swojego projektu i wybierz lokalizację, w której zostanie zapisany.

Konfiguruj nowy projekt

Aplikacja konsoli C# Linux macOS Windows Konsola

Nazwa projektu

Lokalizacja

Nazwa rozwiązania ⓘ

☐ Umieść rozwiązanie i projekt w tym samym katalogu

Informacje dodatkowe

Aplikacja konsoli C# Linux macOS Windows Konsola

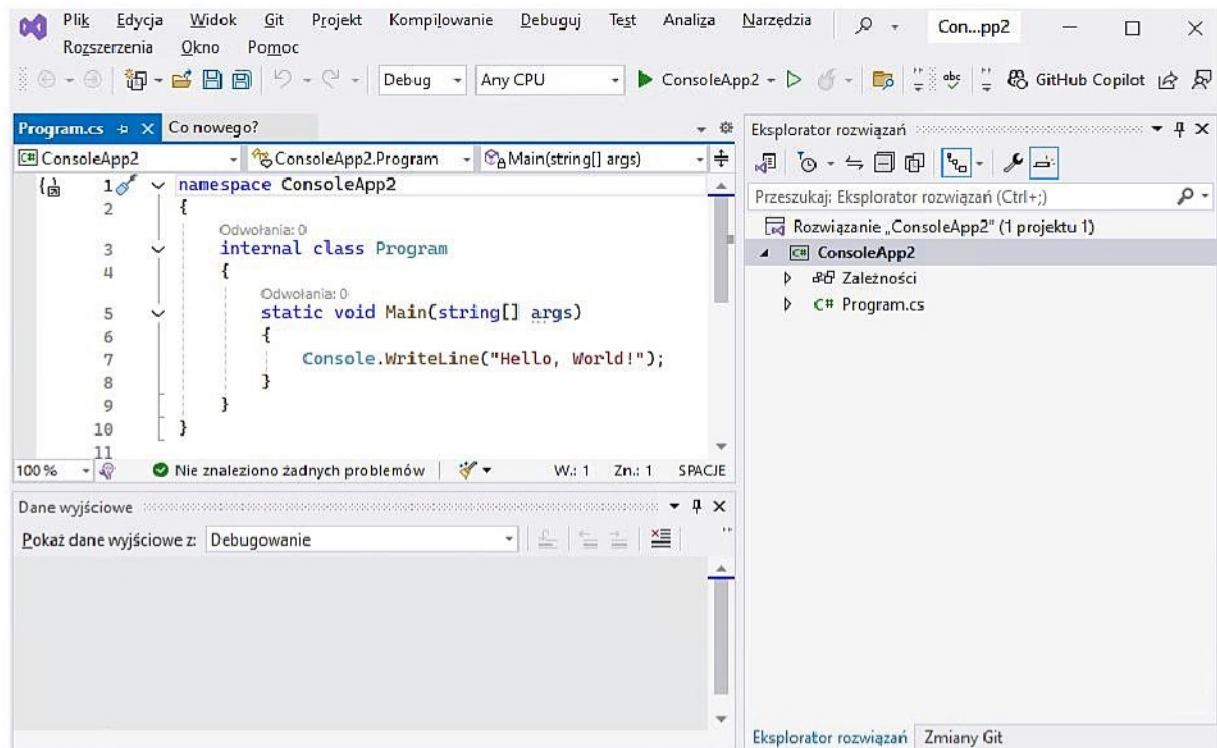
Platforma ⓘ

☒ Nie używaj instrukcji najwyższego poziomu ⓘ

☐ Włącz publikowanie native AOT ⓘ



4. **Utwórz projekt:** Kliknij przycisk "Utwórz".



Gratulacje! Stworzyłeś swój pierwszy projekt w Visual Studio. Możesz teraz zacząć pisać kod w języku C#.

Struktura projektu i programu konsolowego w C# w Visual Studio

Struktura projektu w Visual Studio to sposób organizacji plików i folderów, który ułatwia zarządzanie kodem, zasobami i konfiguracją aplikacji. Dla projektu konsolowego w C# podstawowa struktura jest stosunkowo prosta, ale może się różnić w zależności od dodatkowych funkcji i bibliotek, które wykorzystujesz.

Podstawowe elementy projektu:

- **Plik projektu (.csproj):** Zawiera informacje o projekcie, takie jak odwołania do bibliotek, ustawienia kompilacji i inne konfiguracje.
- **Folder "Properties/Zależności":** Zawiera plik AssemblyInfo.cs, w którym definiuje się informacje o asembli, takie jak nazwa, wersja, opis itp.
- **Plik programu:** Zawiera kod źródłowy głównej klasy programu, zazwyczaj o nazwie Program.cs.

Struktura programu konsolowego:

Program konsolowy w C# zazwyczaj składa się z jednej klasy zawierającej metodę **Main()**, która jest punktem wejścia do programu. Wewnątrz tej metody umieszczamy kod, który ma zostać wykonany.

Przykładowy program "Hello, World!":

```
namespace ConsoleApp
```

```

{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}

```

Wyjaśnienie

using System; Ta linia dodaje przestrzeń nazw System, która zawiera podstawowe klasy i metody używane w programowaniu C#.

namespace ConsoleApp Definiuje przestrzeń nazw dla projektu.

class Program Definiuje klasę Program, która zawiera metodę Main.

static void Main(string[] args): Jest to metoda główna programu, która jest automatycznie wywoływana podczas uruchamiania aplikacji.

Console.WriteLine("Hello, World!"); Ta linia wyświetla tekst "Hello, World!" na konsoli.

Rozszerzenia struktury:

Dodatkowe klasy: W większych projektach możesz tworzyć dodatkowe klasy, aby lepiej organizować kod i zwiększać jego czytelność.

Namespaces: Możesz definiować własne przestrzenie nazw, aby uniknąć konfliktów nazw i lepiej zarządzać dużymi projektami.

Pliki konfiguracyjne: Możesz używać plików konfiguracyjnych (np. appsettings.json) do przechowywania ustawień aplikacji.

Foldery z kodem: Dla większych projektów warto podzielić kod na foldery tematyczne, aby ułatwić nawigację.

Biblioteki zewnętrzne: Możesz dodawać odwołania do bibliotek zewnętrznych, aby korzystać z gotowych funkcjonalności.

Visual Studio ułatwia zarządzanie strukturą projektu:

Eksplorator rozwiązań: Pokazuje wszystkie elementy projektu w hierarchicznej strukturze.

Szablony projektów: Visual Studio oferuje wiele gotowych szablonów projektów, które ułatwiają rozpoczęcie pracy.

Refaktoryzacja: Narzędzia refaktoryzacji pozwalają na zmianę struktury kodu bez wpływu na jego funkcjonalność.

Proste typy danych

Proste typy danych to podstawowe jednostki, które służą do przechowywania pojedynczych wartości. W C# należą do nich:

int: liczby całkowite (np. -10, 0, 42)

double: liczby zmiennoprzecinkowe (np. 3.14, -0.5)

char: pojedynczy znak (np. 'A', '9')

bool: wartość logiczna (true lub false)

string: ciąg znaków (np. "Hello, World!")

Przykład 1: Obliczenie pola koła

```
using System;
namespace ProsteTypyDanych
{
    class Program
    {
        static void Main(string[] args)
        {
            double promien = 5.0; // promień koła (typ double)
            double pi = 3.14159; // liczba pi (typ double)
            double pole = pi * promien * promien; // obliczenie pola (wynik typu double)

            Console.WriteLine("Pole koła wynosi: " + pole);
        }
    }
}
```

Przykład 2: Porównanie dwóch liczb

```
using System;
namespace ProsteTypyDanych
{
    class Program
    {
        static void Main(string[] args)
        {
            int liczba1 = 10;
            int liczba2 = 15;
            bool czySaRowne = liczba1 == liczba2;

            Console.WriteLine("Czy liczby są równe? " + czySaRowne);
        }
    }
}
```

Złożone typy danych

Złożone typy danych pozwalają na grupowanie wielu wartości w jedną strukturę. W C# należą do nich:

Tablice to uporządkowane kolekcje elementów tego samego typu.

Struktury to zbiory pól reprezentujących różne właściwości obiektu.

Klasy to bardziej złożone typy, które mogą zawierać pola, metody, właściwości i zdarzenia.

Przykład 3: Tablica liczb

```
using System;
```

```
namespace ZlozoneTypyDanych
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] liczby = { 1, 2, 3, 4, 5 };
            Console.WriteLine("Pierwszy element tablicy: " + liczby[0]);
        }
    }
}
```

Przykład 4: Struktura Osoba

```
using System;
```

```
namespace ZlozoneTypyDanych
{
    struct Osoba
    {
        public string Imie;
        public string Nazwisko;
        public int Wiek;
    }

    class Program
    {
        static void Main(string[] args)
        {
            Osoba osoba1 = new Osoba();
            osoba1.Imie = "Jan";
            osoba1.Nazwisko = "Kowalski";
            osoba1.Wiek = 30;

            Console.WriteLine("Osoba: " + osoba1.Imie + " " + osoba1.Nazwisko + ", wiek: " +
osoba1.Wiek);
        }
    }
}
```



```
}  
}  
}
```

W języku C# struktura (ang. *struct*) to złożony typ danych, który pozwala na grupowanie różnych zmiennych pod jedną nazwą. W przeciwieństwie do klas, struktury są typami wartości, co oznacza, że przy przypisywaniu jednej struktury do drugiej kopiowana jest cała zawartość, a nie tylko referencja.

```
struct Osoba  
{  
    public string Imie;  
    public string Nazwisko;  
    public int Wiek;  
}
```

struct Osoba Definiuje nową strukturę o nazwie Osoba.

public string Imie; Publiczne pole typu string służące do przechowywania imienia osoby.

public string Nazwisko; Publiczne pole typu string służące do przechowywania nazwiska osoby.

public int Wiek; Publiczne pole typu int służące do przechowywania wieku osoby.

Jak działa ten kod (Przykład 4):

1. **Definicja struktury:** Tworzymy nowy szablon (strukturę) o nazwie Osoba, który opisuje, jakie dane będzie przechowywać.
2. **Tworzenie instancji:**

```
Osoba osoba1 = new Osoba();
```

Tworzymy nową zmienną osoba1 typu Osoba. Słowo kluczowe new służy do alokacji pamięci dla nowej struktury.

3. **Przypisanie wartości:**

```
osoba1.Imie = "Jan";  
osoba1.Nazwisko = "Kowalski";  
osoba1.Wiek = 30;
```

Przypisujemy wartości do poszczególnych pól struktury osoba1.

Zalety stosowania struktur

Struktury są często używane do reprezentowania prostych pojęć, takich jak punkty na płaszczyźnie, daty czy kolory.

Ponieważ struktury są typami wartości, operacje na nich są często bardziej wydajne niż na klasach, zwłaszcza przy przekazywaniu ich jako argumenty do funkcji.

Struktury są niezmiennie, co oznacza, że ich wartości nie mogą być zmienione po utworzeniu. To może pomóc w uniknięciu błędów programistycznych.

Kiedy używać struktur, a kiedy klas?

Struktury:

- Proste typy danych.
- Mała ilość danych.
- Nie wymagają dziedziczenia.
- Nie wymagają złożonych zachowań.

Klasy:

- Złożone typy danych.
- Duża ilość danych.
- Wymagają dziedziczenia.
- Wymagają złożonych zachowań.

Struktura Osoba w ostatnim przykładzie to prosty sposób na reprezentowanie informacji o osobie. Możemy stworzyć wiele takich struktur, każda z własnym zestawem danych. Struktury są użytecznym narzędziem w programowaniu obiektowym, umożliwiając organizację danych w logiczne grupy.

Proste typy danych służą do przechowywania pojedynczych wartości, podczas gdy złożone typy danych pozwalają na grupowanie wielu wartości w bardziej złożone struktury. Wybór odpowiedniego typu danych zależy od rodzaju danych, które chcesz przechowywać i operacji, które chcesz na nich wykonywać.

Zmienne

Zmienne w C# to nazwane obszary w pamięci, które przechowują dane. Każda zmienna ma przypisany określony typ danych (np. int, double, string), który określa rodzaj wartości, jaką może przechowywać.

Przykłady zmiennych

```
int wiek = 30; // Zmienna całkowita przechowująca wiek
string imie = "Jan"; // Zmienna tekstowa przechowująca imię
double wzrost = 1.75; // Zmienna zmiennoprzecinkowa przechowująca wzrost
```

Operacje na zmiennych

Operacje wejścia/wyjścia

Do wprowadzania ciągów z klawiatury w C# używamy metody `Console.ReadLine()`:

Przykład 5: Wczytywanie danych:

```
using System;
namespace WczytywanieDanych
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Podaj swoje imię:");
            string imie = Console.ReadLine();

        }
    }
}
```

Przykład 6: Wyświetlanie danych:

```
using System;
namespace WczytywanieWyswietlanieDanych
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Podaj swoje imię:");
            string imie = Console.ReadLine();
            Console.WriteLine("Witaj, " + imie + "!");

        }
    }
}
```

Operacje arytmetyczne

Dodawanie: +

Przykład 7: Dodawanie:

```
using System;
namespace WczytywanieWyswietlanieDanych
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

int a = 5;
int b = 3;
int wynik = a + b; // wynik = 8
Console.WriteLine("a+b=" + wynik );

    }
}

```

Odejmowanie: -

Mnożenie: *

Dzielenie: /

Modulo (reszta z dzielenia): %

Operacje logiczne

Równość: ==

Różność: !=

Mniejsze niż: <

Większe niż: >

Mniejsze lub równe: <=

Większe lub równe: >=

| logiczne: &&

Lub logiczne: ||

Negacja: !

Przykład 7: Kalkulator prostych działań

Jest to program, który pobiera od użytkownika dwie liczby całkowite, wykonuje na nich podstawowe operacje arytmetyczne (dodawanie, odejmowanie, mnożenie, dzielenie) oraz wyświetla wyniki.

```
using System;
```

```

namespace Kalkulator
{
    class Program
    {
        static void Main(string[] args)

```



```

{
    Console.WriteLine("Podaj pierwszą liczbę:");
    int liczba1 = int.Parse(Console.ReadLine());

    Console.WriteLine("Podaj drugą liczbę:");
    int liczba2 = int.Parse(Console.ReadLine());

    Console.WriteLine("Wybierz działanie (+, -, *, /):");
    char dzialanie = Console.ReadKey().KeyChar;

    int wynik;
    switch (dzialanie)
    {
        case '+':
            wynik = liczba1 + liczba2;
            break;
        case '-':
            wynik = liczba1 - liczba2;
            break;
        case '*':
            wynik = liczba1 * liczba2;
            break;
        case '/':
            wynik = liczba1 / liczba2;
            break;
        default:
            Console.WriteLine("Nieprawidłowe działanie.");
            return;
    }

    Console.WriteLine("Wynik: " + wynik);
}
}
}

```

Wyjaśnienie

- Program wczytuje od użytkownika dwie liczby i znak operacji.
- Następnie, w zależności od wybranej operacji, wykonuje odpowiednie obliczenia i wyświetla wynik.
- Wykorzystuje konstrukcję switch do obsługi różnych przypadków działania.

Podsumowanie

Zmienne są podstawowym elementem każdego programu. Pozwalają na przechowywanie danych, które mogą być następnie przetwarzane za pomocą różnych operacji. Operacje wejścia/wyjścia umożliwiają interakcję z użytkownikiem, operacje

arytmetyczne pozwalają na wykonywanie obliczeń, a operacje logiczne na podejmowanie decyzji w programie.

Zadania

1. Napisz program, który konwertuje temperaturę z Celsjusza na Fahrenheita i odwrotnie. Użytkownik podaje temperaturę i wybiera kierunek konwersji.
2. Prosty kalkulator: Napisz program, który poprosi użytkownika o wprowadzenie dwóch liczb, a następnie wyświetli wynik ich dodawania, odejmowania, mnożenia i dzielenia.
3. Obliczanie pola prostokąta: Napisz program, który poprosi użytkownika o wprowadzenie długości i szerokości prostokąta, a następnie obliczy i wyświetli jego pole.
4. Obliczanie średniej: Napisz program, który oblicza średnią z kilku liczb wprowadzonych przez użytkownika.

Propozycje rozwiązania zadań

1. Konwersja temperatury z Celsjusza na Fahrenheita i odwrotnie

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Wybierz kierunek konwersji: 1 - Celsjusz na Fahrenheit, 2 - Fahrenheit na Celsjusz");
        int wybor = int.Parse(Console.ReadLine());

        Console.WriteLine("Podaj temperaturę:");
        double temperatura = double.Parse(Console.ReadLine());

        if (wybor == 1)
        {
            // Konwersja z Celsjusza na Fahrenheita
            double fahrenheit = (temperatura * 9 / 5) + 32;
            Console.WriteLine($"Temperatura w Fahrenheit: {fahrenheit}");
        }
        else if (wybor == 2)
        {
            // Konwersja z Fahrenheita na Celsjusza
            double celsius = (temperatura - 32) * 5 / 9;
            Console.WriteLine($"Temperatura w Celsjuszach: {celsius}");
        }
        else
        {
            Console.WriteLine("Nieprawidłowy wybór.");
        }
    }
}
```

2. Prosty kalkulator

```
using System;

class Program
{
    static void Main()
```

```

{
    Console.WriteLine("Podaj pierwszą liczbę:");
    double liczba1 = double.Parse(Console.ReadLine());

    Console.WriteLine("Podaj drugą liczbę:");
    double liczba2 = double.Parse(Console.ReadLine());

    // Dodawanie
    double suma = liczba1 + liczba2;
    Console.WriteLine($"Suma: {suma}");

    // Odejmowanie
    double roznica = liczba1 - liczba2;
    Console.WriteLine($"Różnica: {roznica}");

    // Mnożenie
    double iloczyn = liczba1 * liczba2;
    Console.WriteLine($"Iloczyn: {iloczyn}");

    // Dzielenie
    if (liczba2 != 0)
    {
        double iloraz = liczba1 / liczba2;
        Console.WriteLine($"Iloraz: {iloraz}");
    }
    else
    {
        Console.WriteLine("Dzielenie przez zero jest niemożliwe.");
    }
}
}

```

3. Obliczanie pola prostokąta

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj długość prostokąta:");
        double dlugosc = double.Parse(Console.ReadLine());

        Console.WriteLine("Podaj szerokość prostokąta:");
        double szerokosc = double.Parse(Console.ReadLine());

        // Obliczanie pola
        double pole = dlugosc * szerokosc;
        Console.WriteLine($"Pole prostokąta: {pole}");
    }
}

```

4. Obliczanie średniej

```

using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<double> liczby = new List<double>();
        string input;

        Console.WriteLine("Wprowadź liczby (wpisz 'stop', aby zakończyć):");

        while ((input = Console.ReadLine()) != "stop")
        {
            liczby.Add(double.Parse(input));
        }

        // Obliczanie średniej
    }
}

```

```
double suma = 0;
foreach (double liczba in liczby)
{
    suma += liczba;
}

double srednia = suma / liczby.Count;
Console.WriteLine($"Średnia: {srednia}");
}
```